

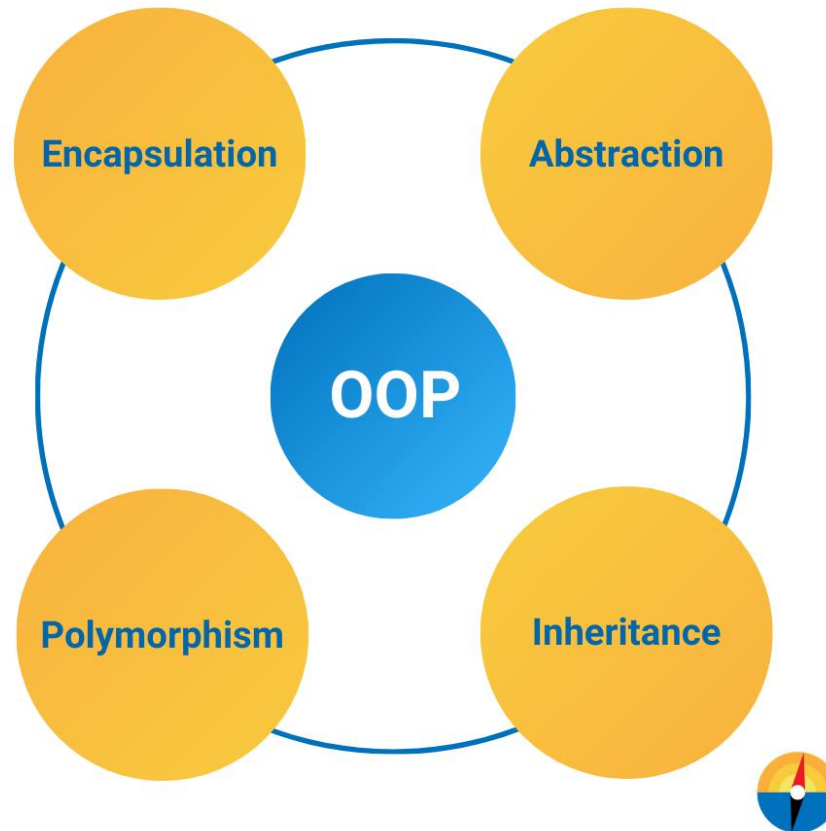
CS-1204
Object-Oriented Programming
(3+1)

Week 03

By

Engr. Muhammad Awais

Pillars of Object-Oriented Programming



Encapsulation in C++

What is Encapsulation?

- Encapsulation is the process of bundling data and methods into a single unit (class).
- It hides internal data from outside interference.
- Data is accessed and modified only through public methods.

Why we use Encapsulation?

- Protect data from accidental or unauthorized changes.
- Control how data is accessed or modified.
- Add validation logic in one place. Simplify future maintenance..

Encapsulation in C++ (cont...)

Real-Life Analogies

- **ATM Machine:** You can't access the balance directly. You use buttons (functions).
- **Car:** You don't touch the engine directly. You use the accelerator, brake, and steering.
- **Medicine Capsule:** You don't see the ingredients—only the final product..

Encapsulation in C++ (cont...)

Achieving Encapsulation

- Make data members **private**.
- Provide public **getter** and **setter** methods.
- **Validate** data inside setters.

```
class Student {  
private:  
    string name;  
    int age;  
  
public:  
    void setName(string n) {  
        if (!n.empty()) {  
            name = n;  
        } else {  
            cout << "Invalid name. Name cannot be empty." << endl;  
        }  
    }  
  
    void setAge(int a) {  
        if (a > 0 && a < 120) {  
            age = a;  
        } else {  
            cout << "Invalid age. Age must be between 1 and 119." << endl;  
        }  
    }  
  
    string getName() { return name; }  
    int getAge() { return age; }  
};
```

Encapsulation in C++ (cont...)

Benefits

- Data members are protected.
- Setters perform validation.
- Code is cleaner, more secure.

```
class Student {  
private:  
    string name;  
    int age;  
  
public:  
    void setName(string n) {  
        if (!n.empty()) {  
            name = n;  
        } else {  
            cout << "Invalid name. Name cannot be empty." << endl;  
        }  
    }  
  
    void setAge(int a) {  
        if (a > 0 && a < 120) {  
            age = a;  
        } else {  
            cout << "Invalid age. Age must be between 1 and 119." << endl;  
        }  
    }  
  
    string getName() { return name; }  
    int getAge() { return age; }  
};
```

Encapsulation in C++ (cont...)

Common Mistakes To Avoid

- Making variables public just to "make it work".
- Skipping validation in setters.
- Writing getters/setters without protecting data.

Encapsulation in C++ (cont...)

Another Example

```
class BankAccount {  
private:  
    double balance;  
  
public:  
    void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        } else {  
            cout << "Invalid deposit amount." << endl;  
        }  
    }  
  
    void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        } else {  
            cout << "Invalid withdraw amount or insufficient funds." << endl;  
        }  
    }  
  
    double getBalance() { return balance; }  
};
```

Task 5: Encapsulation

Create a **Product** class with proper **encapsulation**.

Private data members:

- ❑ string **name**
- ❑ double **price**
- ❑ int **quantity**

Public Methods:

- ❑ void **setName(string n)** - Validate non-empty name.
- ❑ void **setPrice(double p)** - Validate price > 0.
- ❑ void **setQuantity(int q)** - Validate quantity >= 0
- ❑ **Getters** for all fields

Test Code:

```
int main() {  
    Product p;  
    p.setName(""); // Should print error message  
    p.setPrice(-50); // Should print error message  
    p.setQuantity(-2); // Should print error message  
  
    p.setName("Laptop");  
    p.setPrice(1200.50);  
    p.setQuantity(5);  
  
    cout << "Product: " << p.getName() << endl;  
    cout << "Price: $" << p.getPrice() << endl;  
    cout << "Quantity: " << p.getQuantity() << endl;  
    return 0;  
}
```

Rubrics

Criteria	Excellent (3)	Good (2)	Need Improvement (1)	Poor (0)
Class Definition & Object Usage	Correct class structure with object creation; proper syntax.	Minor syntax issues but structure is mostly correct.	Major issues in class or object usage.	Not attempted / completely incorrect.
Member Variables	Relevant attributes defined clearly with proper data types.	Attributes defined with minor issues.	Attributes poorly defined or unclear.	No meaningful use of member variables.
Member Functions (Inside/Outside)	Functions defined and used correctly as per instruction.	Functions defined with small syntax/placement errors.	Poor function structure or logic missing.	No functions defined or irrelevant code.
Use of Access Specifiers	All data members are private/protected and accessed via public methods.	All data members are private/protected and accessed via public methods.	All data members are private/protected and accessed via public methods.	All data members are private/protected and accessed via public methods.
Use of Getters/Setters	Proper getters/setters used with validation and clear naming.	Basic getters/setters present, may lack validation or clarity..	Few or inconsistent getters/setters..	No use of getters/setters; direct access..

REFERENCES

- https://www.w3schools.com/cpp/cpp_oop.asp
- <https://www.programiz.com/cpp-programming/oop>